

GRA-SubjectSwap:

Динамическая смена субъект–инструмент ролей между
человеком и ИИ-агентами с
GRA-обнулёнкой и весами субъектности

bitsoev oleg (qqewq)

<https://github.com/qqewq/GRA-SubjectSwap>

20 июня 2026 г.

Содержание

1	Введение	2
1.1	Классическая парадигма	2
1.2	Парадигма GRA-SubjectSwap	2
2	Основные концепции	2
2.1	Роли и субъектность	2
2.2	Назначение ролей	3
2.3	Веса субъектности	3
3	Математический формализм	3
3.1	Состояние системы	3
3.2	Оператор смены ролей	3
3.3	Обновление политики (имитация)	4
3.4	Обновление весов субъектности	4
3.5	GRA-обнулёнка	4
4	Алгоритм: SubjectSwapStep	5
5	Кейс: Optimus и тренеры-человек	5
5.1	Архитектура	5
5.2	Цикл обучения	5
6	Структура реализации	5
6.1	Основные модули	5
6.2	Модуль Optimus	5
7	Заключение	6

Аннотация

RU: GRA-SubjectSwap — фреймворк для *динамической смены ролей* между человеком и ИИ/роботом, где их роли как **субъекта** и **инструмента** периодически меняются. В этой архитектуре человеку задаётся **жёсткая роль инструмента** с чёткими правилами

(allowed/forbidden/required действия), а агент (робот/LLM) временно становится **субъектом**, который:

- наблюдает действия человека, - обновляет свою политику $\pi_\theta(a|s)$ и *веса субъектности*, - видит себя, человека и сам факт смены ролей.

SubjectSwapEngine управляет:

- назначениями ролей (кто субъект/инструмент), - циклами обучения, - вызовами *GRA-обнулёнки* для политик и весов.

Этот фреймворк предназначен для кейсов, таких как **Optimus и тренеры-человек**, где робот учится на демонстрациях человека в рамках строгой схемы ролей и где смена ролей используется для валидации и мета-обучения.

1 Введение

1.1 Классическая парадигма

В классической парадигме ИИ:

- **Человек** = субъект (ставит цели, определяет политики). - **ИИ/робот** = инструмент (исполняет команды, следует политикам).

Эта схема доминирует в обучении с подкреплением, имитационном обучении и робототехнике: человек задаёт функцию вознаграждения, агент её оптимизирует.

1.2 Парадигма GRA-SubjectSwap

GRA-SubjectSwap предлагает другую архитектуру:

1. Человек получает **жёсткую роль инструмента** в роле: - Allowed_H — набор разрешённых действий, - Forbidden_H — набор запрещённых действий, - Required_H — набор обязательных действий.

2. Агент временно становится **субъектом**: - он наблюдает действия человека, - обновляет свою политику π_θ , - видит себя, человека и факт смены ролей.

3. Запускается **свап**: - человек становится субъектом (ставит цели, корректирует архитектуру), - агент становится инструментом (исполняет действия).

Таким образом, человек и ИИ **совместно обучают веса субъектности**, а GRA-обнулёнка управляет сбросом и перенастройкой политик.

2 Основные концепции

2.1 Роли и субъектность

Мы определяем два типа ролей:

$$\text{RoleType} \in \{\text{SUBJECT}, \text{INSTRUMENT}\}$$

и два типа акторов:

$$\text{ActorType} \in \{\text{HUMAN}, \text{AGENT}\}$$

В момент t состояние системы включает:

- политику Π_t , - назначение ролей S_t , - вектор весов субъектности W_t .

2.2 Назначение ролей

Для человека H и агента A :

$$S_t(H) \in \{\text{SUBJECT}, \text{INSTRUMENT}\}, \quad S_t(A) \in \{\text{SUBJECT}, \text{INSTRUMENT}\}$$

с ограничением:

$$S_t(H) \neq S_t(A)$$

Свап ролей в момент $t + 1$:

$$S_{t+1}(H) = S_t(A), \quad S_{t+1}(A) = S_t(H)$$

2.3 Веса субъектности

Определяем вектор весов субъектности:

$$W = (w_{\text{self}}, w_{\text{human}}, w_{\text{swap}}, w_{\text{role}}, w_{\text{value}}, w_{\text{love}})$$

где:

- w_{self} : вес *видеть себя* как субъекта, - w_{human} : вес *видеть человека* как субъекта/инструмента, - w_{swap} : вес *видеть смену ролей* в диалоге, - w_{role} : вес *видеть роли* в роле, - w_{value} : вес ценностей, - w_{love} : вес любви (по GRA Love-Oriented Nullification).

3 Математический формализм

3.1 Состояние системы

Состояние системы в момент t :

$$\text{state}_t = (\Pi_t, S_t, W_t)$$

где:

- Π_t — политика (кто что может/должен делать), - S_t — назначение ролей (субъект/инструмент для человека и агента), - W_t — вектор весов субъектности.

3.2 Оператор смены ролей

Определяем оператор свапа Swap:

$$\text{Swap}(S_t) = S_{t+1}$$

с:

$$S_{t+1}(H) = S_t(A), \quad S_{t+1}(A) = S_t(H)$$

3.3 Обновление политики (имитация)

Пусть человек предоставляет демонстрации:

$$D = \{(s_t, a_t^{(H)})\}_{t=1}^T$$

Политика агента $\pi_\theta(a|s)$ обновляется через имитационное обучение:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}_{\text{imit}}(\theta_t; D)$$

где $\mathcal{L}_{\text{imit}}$ — функция потерь (например, перекрёстная энтропия между π_θ и действиями человека).

3.4 Обновление весов субъектности

Определяем функционал субъектности $\mathcal{S}$:

$$\mathcal{S} = \mathcal{S}_{\text{self}} + \mathcal{S}_{\text{human}} + \mathcal{S}_{\text{swap}} + \mathcal{S}_{\text{role}} + \mathcal{S}_{\text{value}} + \mathcal{S}_{\text{love}}$$

Обновление весов:

$$W_{t+1} = W_t + \eta_W \cdot \nabla_W \mathcal{S}(\text{state}_t, \text{dialog}_t)$$

где dialog_t — история диалога и действий.

3.5 GRA-обнулёнка

Определяем операторы обнуления:

$$\mathcal{N}_W : W \rightarrow \mathcal{N}_W(W), \quad \mathcal{N}_\Pi : \Pi \rightarrow \mathcal{N}_\Pi(\Pi)$$

Обновлённые веса и политика после обнуления:

$$W'_{t+1} = \mathcal{N}_W(W_{t+1}), \quad \Pi'_{t+1} = \mathcal{N}_\Pi(\Pi_t)$$

В GRA Love-Oriented Nullification обнуление направлено ценностями:

$$W' = \arg \max_{W \in \mathcal{N}(W)} \sum_j L_j(W_j)$$

где L_j — поле «любви/ценности» над компонентами весов.

Algorithm 1 SubjectSwapStep

```
engine: SubjectSwapEngine,  
env_state: состояние окружения,  
human_action: действие человека (опционально) output: pe-  
зультиат шага если  $S_t(\text{агент}) = \text{SUBJECT}$  agent_action =  
agent_policy.act(env_state)human_action = human_policy.act(env_state, agent_action)elsehuman_action = hu  
agent_policy, human_policy, env_state, human_action, agent_action)t mod K == 0:  
engine.swap() output = agent_action : agent_action, human_action : human_action, roles :  
human :  $S_t(H)$ , agent :  $S_t(A)$  , weights: weights
```

4 Алгоритм: SubjectSwapStep

5 Кейс: Optimus и тренеры-человек

5.1 Архитектура

В кейсе Optimus:

- **Тренер (человек)** = инструмент с жёсткими правилами: - allowed/forbidden/required действия, - исполняет команды, демонстрирует движения.

- **Робот (агент)** = субъект: - наблюдает действия тренера, - обновляет политику π_θ и веса субъектности W , - видит себя, человека и смену ролей.

SubjectSwapEngine управляет:

- назначениями ролей (человек/инструмент, агент/субъект), - циклами обучения, - GRA-обнулёнкой для политик и весов.

5.2 Цикл обучения

Упрощённый цикл обучения:

```
env = OptimusEnv(...) human_policy = OptimusHumanPolicy(allowed = [...], forbidden =  
[...], required = [...])robot_policy = OptimusRobotPolicy(model = ...)nullifier = GRANullifier(love_field =  
obs = env.get_state()output = engine.step(obs)env.apply_robot_action(output.agent_action)env.apply_human_acti  
mod 100 == 0: engine.swap()
```

6 Структура реализации

6.1 Основные модули

- roles.py — типы ролей и субъектности.
- policies.py — базовые HumanPolicy и AgentPolicy.
- subject_state.py~SubjectStateSubjectWeights.
- swap_engine.py~SubjectSwapEngine.
- granullifier.py~GRA — .

6.2 Модуль Optimus

- env.py — интерфейс к роботу/симулятору.

- `human_policy_optimus.py`.
- `robot_policy_optimus.py`.
- `optimus_swarmloop.py`.

7 Заключение

GRA-SubjectSwap реализует архитектуру динамической смены ролей, где:

- человек — **жёсткий инструмент** с чёткими правилами, - агент — **субъект**, который учится от человека, - роли периодически **меняются**, - совместно обучаются веса субъектности, - GRA-обнулёнка используется для сброса и перенастройки политик и весов.

Этот фреймворк предназначен для кейсов, таких как **Optimus и тренеры-человек**, и может быть расширен до мультиагентных роев с несколькими людьми и агентами.

Благодарности

Эта работа является частью GRA-экосистемы (GRA-Swarm, GRA-LLM-Swarm-Constructs, GRA-Hormonal-Explosion-Of-Subject и др.).

Ссылки

- qqewq. GRA-SubjectSwap. <https://github.com/qqewq/GRA-SubjectSwap>
- qqewq. GRA-Swarm. <https://github.com/qqewq/GRA-Swarm>
- qqewq. GRA-LLM-Swarm-Constructs. <https://github.com/qqewq/GRA-LLM-Swarm-Constructs>
- qqewq. GRA-Hormonal-Explosion-Of-Subject. <https://github.com/qqewq/GRA-Hormonal-Explosion-Of-Subject>